

EXPERIMENT-3

Name:- Julakanti Sravani

Roll No:- 241210052

Branch:- CSE-1

Subject:- Computer Architecture and Organization

Subject code:- CSBB 251

Aim:- To view program execution in memory of the computer using GDB compiler

Software tools used:- GDB Compiler, terminal (nano editor)

Theory:-

GDB (GNU Debugger) is a tool that helps you:

- Run your program step-by-step
- Inspect variables and memory
- Find and fix errors (bugs)

Memory Layout of a Program

A program in memory is divided into:

- **Text Segment** → Stores program code
- **Data Segment** → Global & static variables
- **Heap** → Dynamic memory (malloc, calloc)
- **Stack** → Local variables, function calls
- Stack grows downward
- Heap grows upward

Pointers and Memory

- **&** → Gives **address of variable**
- ***** → Gives **value at that address**

Example:

```
int a = 5;  
int *p = &a;
```

- `p` stores address of `a`
- `*p` gives value of `a`

Important Registers in GDB

- `$rip` → Instruction Pointer (current instruction)
- `$rsp` → Stack Pointer (top of stack)
- `$rbp` → Base Pointer (stack frame base)

GDB Commands:-

1. `list` – Displays the source code
2. `break` – Sets a breakpoint in the program
3. `run` – Starts execution of the program
4. `next` – Executes the next line
5. `step` – Steps into a function
6. `print` – Displays the value of variables
7. `x` – Examines memory contents
8. `continue` – Resumes execution of the program
9. `disassemble` – Shows assembly-level instructions

Procedure:-

1. Write the program (create a C program).
2. Compile with debug option

```
gcc -g program.c -o program
```

3. Start GDB

```
gdb ./program
```

4. Set breakpoint

```
break main
```

5. Run the program

```
run
```

6. Execute step-by-step

```
next // next line
```

```
step // go inside function
```

7. Check variables

```
print a
```

```
print p
```

8. View memory

```
x/4x &a (Displays memory content)
```

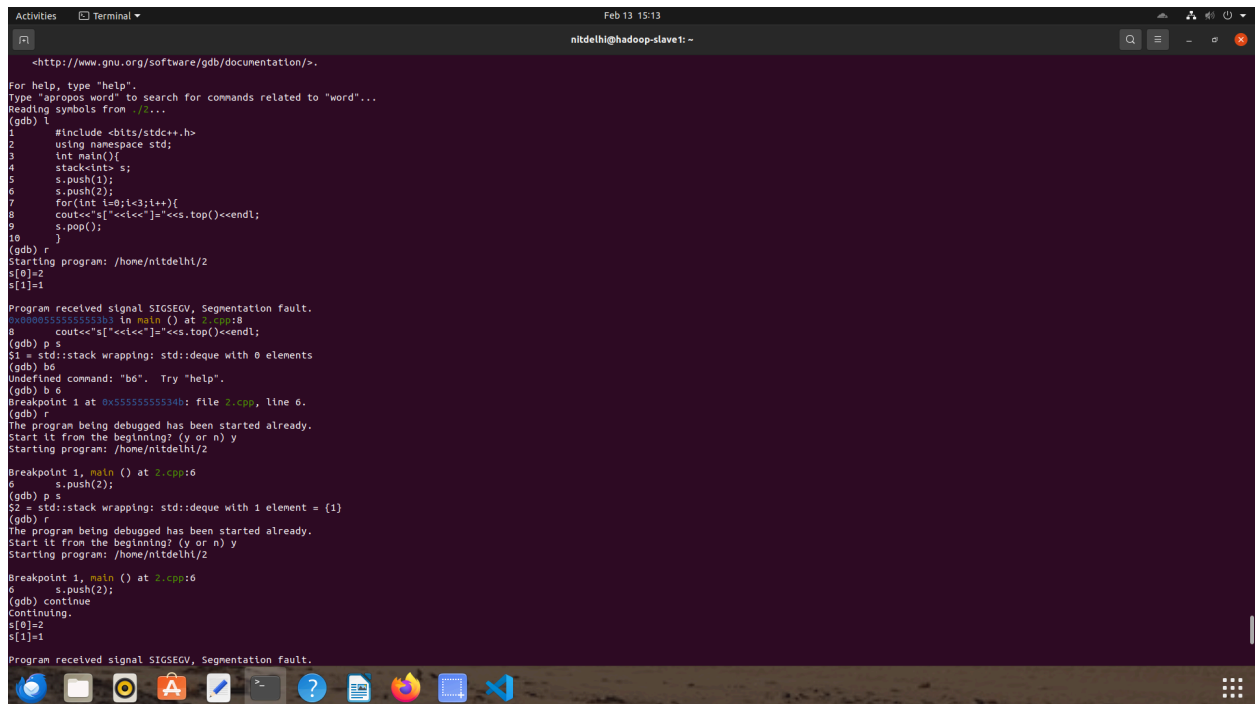
9. Continue execution

```
continue
```

10. Exit GDB *quit*

1.

2.



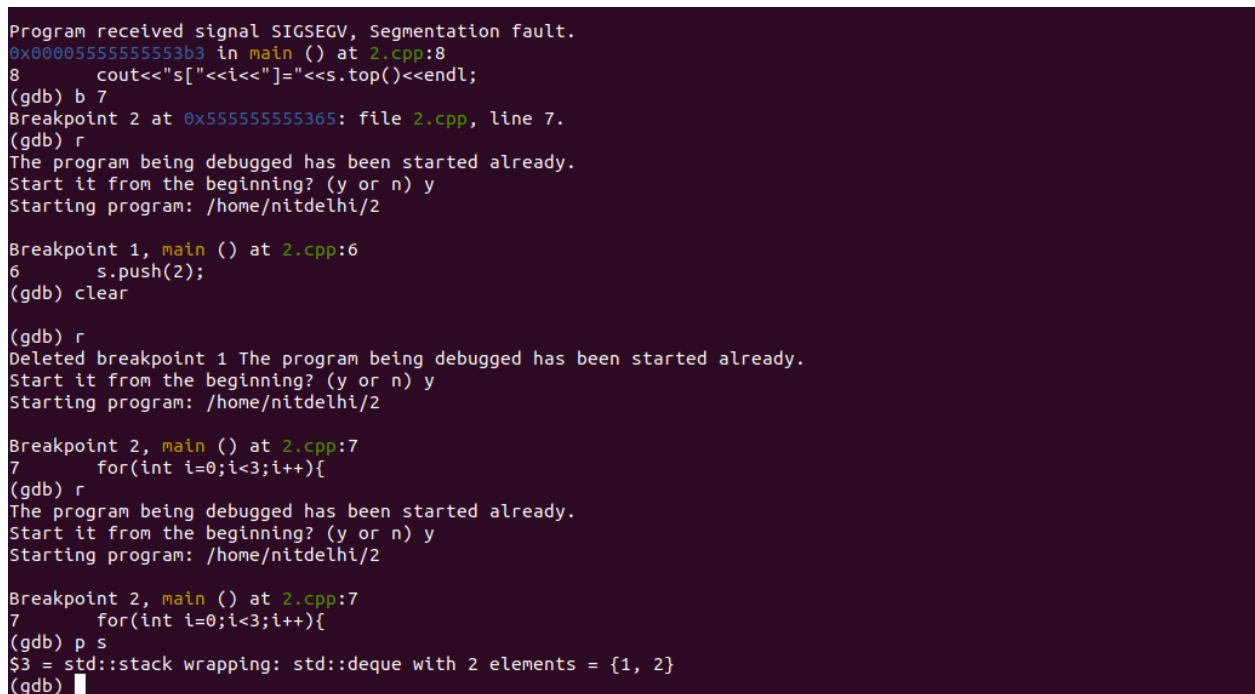
```
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./2...
(gdb) l
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main(){
4  stack<int> s;
5  s.push(1);
6  s.push(2);
7  for(int i=0;i<3;i++){
8  cout<<"s["<<i<<"]="<<s.top()<<endl;
9  s.pop();
10 }
(gdb) r
Starting program: /home/nitdelhi/2
s[0]=2
s[1]=1

Program received signal SIGSEGV, Segmentation fault.
0x00005555555553b3 in main () at 2.cpp:8
8  cout<<"s["<<i<<"]="<<s.top()<<endl;
(gdb) p s
s = std::stack wrapping: std::deque with 0 elements
(gdb) b 6
Undefined command: "b6". Try "help".
(gdb) b 6
Breakpoint 1 at 0x55555555534b: file 2.cpp, line 6.
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/nitdelhi/2

Breakpoint 1, main () at 2.cpp:6
6  s.push(2);
(gdb) p s
s = std::stack wrapping: std::deque with 1 element = {1}
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/nitdelhi/2

Breakpoint 1, main () at 2.cpp:6
6  s.push(2);
(gdb) continue
Continuing.
s[0]=2
s[1]=1

Program received signal SIGSEGV, Segmentation fault.
```



```
Program received signal SIGSEGV, Segmentation fault.
0x00005555555553b3 in main () at 2.cpp:8
8  cout<<"s["<<i<<"]="<<s.top()<<endl;
(gdb) b 7
Breakpoint 2 at 0x555555555365: file 2.cpp, line 7.
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/nitdelhi/2

Breakpoint 1, main () at 2.cpp:6
6  s.push(2);
(gdb) clear

(gdb) r
Deleted breakpoint 1 The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/nitdelhi/2

Breakpoint 2, main () at 2.cpp:7
7  for(int i=0;i<3;i++){
(gdb) r
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/nitdelhi/2

Breakpoint 2, main () at 2.cpp:7
7  for(int i=0;i<3;i++){
(gdb) p s
s3 = std::stack wrapping: std::deque with 2 elements = {1, 2}
(gdb)
```

Result:-

The execution of the program was successfully analyzed using GDB, and memory contents along with variable values were observed.

